Oop in dart

Why do we use oop?
-It keeps code organized, not chaotic.
- Reusability
- Easier to maintain & update.

In one sentence
OOP exists to make large, complex programs easier to understand, extend, and maintain.

First concept:
- Class : what an object has (variables / properties)
- what an object can do (functions / methods)

An object is the actual thing created from that blueprint.

```
class Person {
  String name = "";
  int age = 0;

  void sayHello() {
    print("Hi, my name is $name and I am
$age years old");
  }
```

And in the main function //

```
void main() {
  Person p1 = Person();

p1.name = "Yasmine";
  p1.age = 19;

  p1.sayHello();
}
```

Second concept we have is constructor: is a special function inside a class that runs automatically when you create an object from that class.

```
class Person {
  String name;
  int age;

 Person(this.name, this.age);  //it has the same name as class
}}
```

After we made a constructor, now we can't run the code till we give name and ages values .

Dart won't let you do more than constructor in the same class with the

same name so we have the named constructor :
is a constructor with a name after the class name.
It lets you create different ways to create the same object.

ClassName.constructorName()

Encapsulation
Encapsulation means:
hiding data + controlling access to it
You don’t let anyone touch your data directly.
You decide how it can be read or changed.

```
class BankAccount {
  double _balance = 0; // private

  double get balance {
    return _balance;
  }

  void deposit(double amount) {
    if (amount > 0) {
      _balance += amount;
    }
  }

  void withdraw(double amount) {
    if (amount > 0 && amount <= _balance) {
      _balance -= amount;
    }
  }
}
```

In the main function :

```
void main() {
  BankAccount acc = BankAccount();

  acc.deposit(500);
  acc.withdraw(200);

  print(acc.balance); // 300
}
```

Inheritance
a class inherits properties and methods
from another class
Single Inheritance
One parent → one child

```
class Animal {
  void eat() {
```

```
    print("Animal eats");
  }
}

class Dog extends Animal {
  void bark() {
    print("Dog barks");
  }
}

void main() {
  Dog d = Dog();
  d.eat();   // inherited
  d.bark();  // own
```

Multilevel Inheritance

Parent → child → grandchild

```
class Animal {
  void eat() {
    print("Animal eats");
  }
}
```

```
class Mammal extends Animal {
  void walk() {
    print("Mammal walks");
  }
}

class Dog extends Mammal {
  void bark() {
    print("Dog barks");
  }
}

void main() {
  Dog d = Dog();
  d.eat();   // from Animal
  d.walk();  // from Mammal
  d.bark();  // from Dog
}
```

--

Hierarchical Inheritance
One parent → multiple children

```
class Animal {
  void eat() {
    print("Animal eats");
  }
}
```

```
class Dog extends Animal {
  void bark() {
    print("Dog barks");
  }
}

class Cat extends Animal {
  void meow() {
    print("Cat meows");
  }
}

void main() {
  Dog d = Dog();
  Cat c = Cat();
d.eat();
d.bark();
c.eat();
c.meow();}
```

Polymorphism

It means object can has many shapes.

Class humans { void printclass (){
print("humans");}}
Class manger extend humans {
//here we're going to give the method
another shape

@override void printclass(){
print("manger");}}
Got it??